

- Description

This module manages a smart factory energy consumption monitoring and efficiency optimization system, ensuring that:

1. **The power usage of different machines is continuously monitored to detect excessive consumption.**
2. **If total energy consumption exceeds a predefined threshold, non-essential systems are temporarily shut down to reduce waste.**
3. **If energy usage is below an efficiency threshold, machines are optimized for better power utilization.**
4. **A log of all energy readings, shutdown actions, and efficiency improvements is maintained for industrial performance tracking.**

This system helps prevent energy wastage, improve factory efficiency, and optimize power distribution for sustainable operations.

- Specification

module SmartFactoryEnergyManagement

var currentEnergyUsage, maxEnergyThreshold, minEfficiencyThreshold: real

var powerSavingMode: boolean

var log: sequence of string

process updateEnergyUsage(changeAmount: real)

functional scenario form:

currentEnergyUsage >= minEfficiencyThreshold && log = log $\oplus$ ("Energy Usage Updated: " + currentEnergyUsage + " kWh, Status Normal."), Energy usage updated successfully.

||

currentEnergyUsage > maxEnergyThreshold && powerSavingMode = true, log = log $\oplus$ ("High Energy Warning: " + currentEnergyUsage + " kWh, Activating Power-Saving Mode."), High energy warning triggered, power-saving mode activated.

||

currentEnergyUsage < minEfficiencyThreshold && powerSavingMode = false, log = log $\oplus$ ("Low Efficiency Alert: " + currentEnergyUsage + " kWh, Optimizing Machine Operation."), Low efficiency alert triggered, machine optimization applied.

end process

end module

- Java Code

```
package code;
import java.util.*;
class FactoryEnergySystem {
    private double maxEnergyThreshold;
    private double minEfficiencyThreshold;
    private double currentEnergyUsage;
    private boolean powerSavingMode;
    private List<String> log;
```

```

    public FactoryEnergySystem(double maxEnergyThreshold, double minEfficiencyThreshold, double initialEnergyUsage) {
        this.maxEnergyThreshold = maxEnergyThreshold;
        this.minEfficiencyThreshold = minEfficiencyThreshold;
        this.currentEnergyUsage = initialEnergyUsage;
        this.powerSavingMode = false;
        this.log = new ArrayList<>();
    }

    public void updateEnergyUsage(double changeAmount) {
        currentEnergyUsage += changeAmount;
        manageEnergyEfficiency();
        String message = String.format("Current Energy Usage: %.2f kWh", currentEnergyUsage);
        if (currentEnergyUsage > maxEnergyThreshold) {
            message += " | WARNING: High Energy Consumption! Activating Power-Saving Mode.";
        } else if (currentEnergyUsage < minEfficiencyThreshold) {
            message += " | ALERT: Low Efficiency! Optimizing Machine Operation.";
        } else {
            message += " | Energy Usage is Normal.";
        }
        System.out.println(message);
        log.add(message);
    }

    private void manageEnergyEfficiency() {
        if (currentEnergyUsage > maxEnergyThreshold) {
            powerSavingMode = true;
        } else {
            powerSavingMode = false;
        }
    }

    public void printLog() {
        System.out.println("\nFactory Energy Consumption Log:");
        for (String logEntry : log) {
            System.out.println(logEntry);
        }
    }
}

public class c47_SmartFactorySystem {
    public static void main(String[] args) {
        FactoryEnergySystem factory = new FactoryEnergySystem(5000.0, 1000.0, 3000.0);
        // testcase-VT:

```

```
//      factory.updateEnergyUsage(1200.0);
//      factory.updateEnergyUsage(-1500.0);
//      factory.updateEnergyUsage(-1800.0);
//      factory.updateEnergyUsage(2500.0);
//      factory.updateEnergyUsage(1800.0);
//      factory.printLog();

//      // testcase-FT:
//      factory.updateEnergyUsage(2.3292275018647134E293);
//      factory.updateEnergyUsage(-1.7976931348623157E308);
//      factory.updateEnergyUsage(1.7976931348623157E308);
//      factory.updateEnergyUsage(0.0);
//      factory.updateEnergyUsage(6.838283689476259E307);
//      factory.printLog();

//      // testcase-Z3:
//      factory.updateEnergyUsage(1200);
//      factory.updateEnergyUsage(-1500);
//      factory.updateEnergyUsage(-1800);
//      factory.updateEnergyUsage(2500);
//      factory.updateEnergyUsage(1800);
//      factory.printLog();

//      // testcase-UVT:
//      factory.updateEnergyUsage(1200.0);
//      factory.updateEnergyUsage(-1500.0);
//      factory.updateEnergyUsage(-1800.0);
//      factory.updateEnergyUsage(2500.0);
//      factory.updateEnergyUsage(1800.0);
//      factory.printLog();
}
}
```

- Coverage report

1. Test case number: 5
2. V-Method testing branch coverage: 100%

Element	Coverage	Covered Branches	Missed Branches	Total Branches
▼ VibrationTestingTool	1.0 %	8	834	842
▼ src	1.0 %	8	834	842
▼ code	1.0 %	8	834	842
▼ c47_SmartFactorySystem.java	100.0 %	8	0	8
> c47_SmartFactorySystem		0	0	0
> FactoryEnergySystem	100.0 %	8	0	8

3. Vibration testing instruction coverage: 97.7%

Element	Coverage	Covered Instructions	Missed Instructions	Total Instructions
✓ VibrationTestingTool	0.6 %	129	21,283	21,412
✓ src	0.6 %	129	21,283	21,412
✓ code	0.6 %	129	21,283	21,412
✓ c47_SmartFactorySystem.java	97.7 %	129	3	132
> c47_SmartFactorySystem	89.3 %	25	3	28
> FactoryEnergySystem	100.0 %	104	0	104

4. Fuzz testing branch coverage: 87.5%

Element	Coverage	Covered Branches	Missed Branches	Total Branches
✓ VibrationTestingTool	0.8 %	7	835	842
✓ src	0.8 %	7	835	842
✓ code	0.8 %	7	835	842
✓ c47_SmartFactorySystem.java	87.5 %	7	1	8
> c47_SmartFactorySystem		0	0	0
> FactoryEnergySystem	87.5 %	7	1	8

5. Fuzz testing instruction coverage: 95.5%

Element	Coverage	Covered Instructions	Missed Instructions	Total Instructions
✓ VibrationTestingTool	0.6 %	126	21,286	21,412
✓ src	0.6 %	126	21,286	21,412
✓ code	0.6 %	126	21,286	21,412
✓ c47_SmartFactorySystem.java	95.5 %	126	6	132
> c47_SmartFactorySystem	89.3 %	25	3	28
> FactoryEnergySystem	97.1 %	101	3	104

6. Z3 branch coverage: 100%

Element	Coverage	Covered Branches	Missed Branches	Total Branches
✓ VibrationTestingTool	1.0 %	8	834	842
✓ src	1.0 %	8	834	842
✓ code	1.0 %	8	834	842
✓ c47_SmartFactorySystem.java	100.0 %	8	0	8
> c47_SmartFactorySystem		0	0	0
> FactoryEnergySystem	100.0 %	8	0	8

7. Z3 instruction coverage: 97.7%

Element	Coverage	Covered Instructions	Missed Instructions	Total Instructions
✓ VibrationTestingTool	0.6 %	129	21,283	21,412
✓ src	0.6 %	129	21,283	21,412
✓ code	0.6 %	129	21,283	21,412
✓ c47_SmartFactorySystem.java	97.7 %	129	3	132
> c47_SmartFactorySystem	89.3 %	25	3	28
> FactoryEnergySystem	100.0 %	104	0	104

8. Unimproved V-method branch coverage: 100%

Element	Coverage	Covered Branches	Missed Branches	Total Branches
✓ VibrationTestingTool	1.0 %	8	834	842
✓ src	1.0 %	8	834	842
✓ code	1.0 %	8	834	842
✓ c47_SmartFactorySystem.java	100.0 %	8	0	8
> c47_SmartFactorySystem		0	0	0
> FactoryEnergySystem	100.0 %	8	0	8

9. Unimproved V-method instruction coverage: 97.7%

Element	Coverage	Covered Instructions	Missed Instructions	Total Instructions
✓ VibrationTestingTool	0.6 %	129	21,283	21,412
✓ src	0.6 %	129	21,283	21,412
✓ code	0.6 %	129	21,283	21,412
✓ c47_SmartFactorySystem.java	97.7 %	129	3	132
> c47_SmartFactorySystem	89.3 %	25	3	28
> FactoryEnergySystem	100.0 %	104	0	104